Integration checklist

Run this on the integration day (13 October 2026) before merging anything into `main`. Every box must be checked; an unchecked box blocks the merge. Tick a box only after running the command or looking at the output, not from memory.

Before the merge

- [ ] the capture stage CI is green for the current head. `gh run list --repo ForgeLabHub/ForgeMadrigalProtocolLab --branch capture --limit 3`
- [ ] the protocol stage CI is green for the current head. `gh run list --repo ForgeLabHub/ForgeMadrigalProtocolLab --branch protocol --limit 3`
- [ ] The protocol stage has merged the current `main` locally and the suite is green there, so the merge is a fast-forward of history, not a surprise. `git checkout protocol && git merge main && pytest tests/ -q`
- [ ] The files each branch changed in the shared directories do not overlap, or the overlap is understood before starting. Shared locations are `docs/`, `README.md`, `ARCHITECTURE.md`, `.gitignore`, and `.github/workflows/`. `git diff --name-only main...capture > /tmp/a1.txt && git diff --name-only main...protocol > /tmp/a2.txt && comm -12 <(sort /tmp/a1.txt) <(sort /tmp/a2.txt)`
- [ ] `docs/CONTRACT.md` and both schemas are unchanged by either branch. `git diff main -- docs/CONTRACT.md docs/schemas/`

During the merge

- [ ] Resolve every conflict by hand. Never accept "ours" or "theirs" for a whole file, and never push a merge without reading the resolved diff. `git --no-pager diff --check`
- [ ] A conflict in any of these shared files is escalated to the participant, who decides; it is not resolved unilaterally: `docs/CONTRACT.md`, `.gitignore`, `README.md`, `ARCHITECTURE.md`, `.github/workflows/ci.yml`.
- [ ] The merged `.github/workflows/ci.yml` keeps the Python and OS matrix, the code-quality job, and the audit job from both branches.
- [ ] After resolving, `git --no-pager diff main..HEAD --stat` lists only the expected files.

After the merge

- [ ] Full suite on `main`: `pytest tests/ -q` is green.
- [ ] CI on `main` is green. `gh run list --repo ForgeLabHub/ForgeMadrigalProtocolLab --branch main --limit 3`
- [ ] The pipeline runs end to end on the corpus: `python -m src.project.cli pipeline --pcap tests/corpus/corpus_capture_01.pcapng --project project.madrigal --report reports/REPORT.md`
- [ ] The GUI opens `reference_export/corpus_capture_01.normalized.json` (or the matching project result) and lists three sessions.
- [ ] Every file in `tests/corpus/reference_export/` validates against `docs/schemas/capture.schema.json`. `python -m pytest tests/integration -q`
- [ ] A project opens after being copied to another directory. `cp -r project.madrigal /tmp/ && python -c "from src.project import Project; Project.open('/tmp/project.madrigal')"`

- [ ] If the protocol branch adds a reference investigation document (for example `docs/REFERENCE_INVESTIGATION.md`), its figures match `REPORT.md`: the same counterexample examples, the same message counts, and the same corpus coverage. A mismatch in any figure blocks the merge until the two documents are reconciled. Read the numbers from both documents; do not carry them in this checklist.

Reference export

- [ ] The three files exist in `tests/corpus/reference_export/`.
- [ ] Each one validates against the capture schema. `python -m pytest tests/integration -q`
- [ ] The exports still match the module. `bash tests/corpus/reference_export/regenerate.sh && git diff --stat`

Protocol stage

- [ ] Applying `corpus_rule_v1.json` to every file in `reference_export/` completes without an error and produces a result that validates against `docs/schemas/result.schema.json`.
- [ ] On `corpus_capture_defects.normalized.json`, the messages that fall on the `gap` and the `ambiguity` are classified `incomplete` or `ambiguous`, never `matched`.
- [ ] The rule finds at least one counterexample on `corpus_capture_02` (same protocol, different values).
- [ ] Changing the rule raises its version and marks the previous result `outdated`.

GUI stage

- [ ] The GUI opens `reference_export/corpus_capture_01.normalized.json` and lists three sessions.
- [ ] The hex view renders the stream bytes and highlights the diagnostics on the defects file.
- [ ] Clicking a byte shows its packet index, sequence number, and timestamp from the provenance entry.
- [ ] Clicking a counterexample jumps to the matching byte in the hex view.

End-to-end

- [ ] The full path runs on one capture: PCAP -> capture -> protocol -> GUI -> report. `python -m src.capture.cli --pcap tests/corpus/corpus_capture_01.pcapng --out out/cap01.json`
- [ ] A report renders from the investigation: `python -m pytest tests/report -q` and the HTML opens without network.
- [ ] The project inside `project.madrigal/` opens after being copied to `/tmp`. `cp -r project.madrigal /tmp/ && python -c "from src.project import Project; Project.open('/tmp/project.madrigal')"`
- [ ] A project exports to a zip and imports back with matching digests.

Documentation

- [ ] `README.md` describes the current commands and points at the right docs.
- [ ] `ARCHITECTURE.md` matches what was built, including what is not done.
- [ ] `docs/CAPTURE_API.md`, `docs/PROTOCOL_API.md`, and `docs/INTEGRATION.md` are current.
- [ ] `REPORT.md` reflects the investigation actually performed.

- [ ] No `TODO` or `FIXME` remains, and no file names a tool, model or assistant. The signature regex lives in the report guard, so this check does not spell the names out. `git grep -nE "TODO|FIXME"` and `pytest tests/integration/test_reference_report.py -k automation_traces`

Environment

- [ ] The suite passes in a clean container on both ends of the supported range. `scripts/test_in_docker.sh ubuntu:22.04 3.12` and `scripts/test_in_docker.sh ubuntu:24.04 3.13`
- [ ] A fresh clone installs and runs without the development host: `python3 -m venv .venv && source .venv/bin/activate && pip install -e ".[dev]" && pytest tests/ -q`
- [ ] The results in `docs/PORTABILITY.md` match the run just performed.

Submission

- [ ] The submission archive builds and verifies from a clean tree: `python -m scripts.build_submission`
- [ ] The archive holds `SUBMISSION.md`, the sources, the tests and corpus, the docs, and the presentation, and no cache or virtual environment. `python -c "import zipfile,glob; print(zipfile.ZipFile(sorted(glob.glob('dist/submission_*.zip'))[-1]).namelist())"`
- [ ] The archive reproduces the suite after extraction; the build reports "archive verified".

Full test run

- [ ] `pytest tests/ -q` is green on `main` after the merges.
- [ ] No capture, project directory, or `__pycache__` is committed. `git status --short`